

CC4M: Code Clone Analysis and Visualization for Microservices

Gen Kawamata, Yuki Ota, Norihiro Yoshida, Shiyu Yang, Erina Makihara, Katsuro Inoue
Ritsumeikan University

Osaka, Japan

{is0665sp, is0607ih}@ed.ritsumei.ac.jp, {norihiro, yangsy, makihara, inoue-k}@fc.ritsumei.ac.jp

Abstract—Microservice architecture supports software evolution by decomposing a system into small, loosely coupled services that can be deployed independently. Contrary to the expectation of high modularity, prior studies have reported that code clones exist across service boundaries, some of which are co-modified in the same version. Such clones may require changes to be propagated across service boundaries, thereby undermining service independence and increasing maintenance costs. However, existing tools do not support microservice-aware clone analysis. We present CC4M, a microservice-aware clone analysis and visualization tool. CC4M detects and enriches clone pairs with service-boundary, co-modification, file-category, and metric information. The enriched clones are visualized in an interactive scatter plot with explicit service boundaries, supporting metric-based filtering to prioritize clones with potentially higher maintenance impact. Using an open-source microservice application, we illustrate how CC4M helps identify the potential impact scope of code changes. A demo video and the tool are available at <https://www.youtube.com/watch?v=0xOIQPFbkUg> and <https://github.com/gg117e/CC4M>, respectively.

Index Terms—microservices, code clone, visualization

I. INTRODUCTION

Microservice architecture decomposes a software system into small, loosely coupled, and independently deployable services. It has been widely adopted in both open-source and industrial projects, including systems at Netflix, Amazon, and Spotify, to support software evolution through modularity and technology heterogeneity [1]–[3].

Because microservices emphasize modularity and service independence, one might expect clones across service boundaries to be limited. However, Mo et al. reported that code clones exist across microservices in open-source microservice projects, and showed that clones are co-modified across service boundaries between consecutive releases [4]. Here, code clones are identical or similar code fragments [5]; a clone pair is a pair of such fragments, and a clone set is a set of mutually similar fragments. Co-modification refers to the case in which the code fragments constituting a clone pair are modified together in the project’s revision history. Following prior work [4], we classify clone pairs in microservice systems based on service boundaries. A clone pair is called a *within-service clone pair* when both fragments belong to the same service, and a *cross-service clone pair* when the two fragments belong to different services [4]. In subsequent work, Zhao et al. analyzed cross-service clones based on file categories and

showed that the distribution and underlying causes of clones differ across categories [6].

These findings indicate that cross-service clones exist, are co-modified, and concentrate in specific file categories, suggesting that such clones may propagate changes across service boundaries and undermine service independence. However, existing clone detection and visualization tools [7]–[9] organize clone information by file system or program structure (e.g., directories, files, and classes) rather than by service boundaries. In particular, they lack two capabilities: (i) service-boundary awareness, so within-service and cross-service clone pairs are not distinguished and the scope of change impact is unclear; and (ii) co-modification awareness, so historical change coupling is not integrated, hindering the identification of clones that affect maintainability.

To address these limitations, we introduce CC4M, a tool for detecting and visualizing code clones in microservice systems. It detects Type-2 clones with CCFinderSW, a code clone detector that supports multiple programming languages [10], and identifies service boundaries with CLAIM [11]. CC4M then enriches each clone pair with service boundary, co-modification, and file-category information, the last of which is inspired by the file-level analysis of Zhao et al. [6]. Based on the enriched information, CC4M computes clone metrics, such as service span and co-modification frequency. The enriched clone pairs are visualized through three views: a *Statistics View* that summarizes project-wide clone statistics, an interactive *Scatter Plot* with explicit service boundaries, and a *Metric View* for metric-based prioritization and drill-down across services, clone sets, and files.

The contributions of this paper are as follows. First, we present CC4M, a microservice-aware clone analysis tool that enriches clone pairs with service-boundary, co-modification, and file-category information. Second, we introduce clone metrics, such as service span and co-modification frequency, to quantify the impact scope and change coupling across service boundaries. Third, we provide a scatter plot with explicit service boundaries and a *Metric View* for metric-based prioritization and drill-down.

II. CC4M

Fig. 1 shows an overview of CC4M. It takes as input the GitHub URL of a Dockerized microservice system and

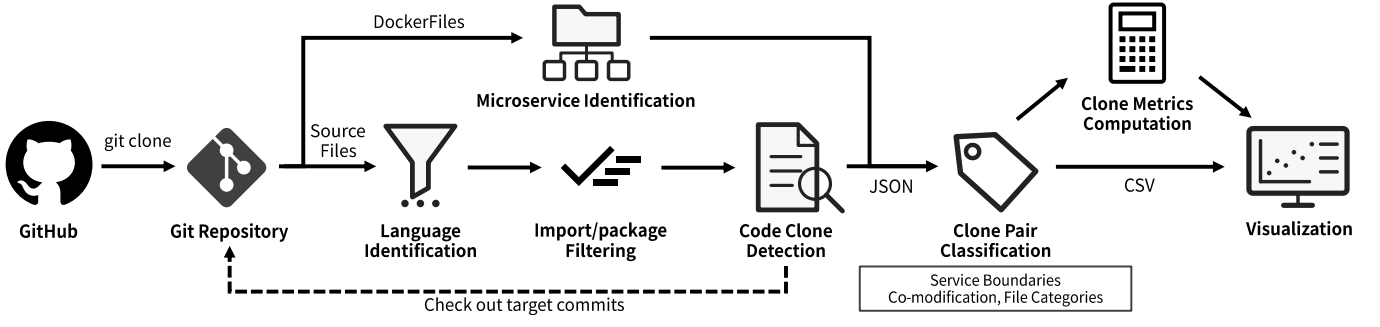


Fig. 1. An overview of CC4M. It identifies microservices from a Git repository, detects code clones, enriches them with service boundaries, co-modification history, and file categories, computes clone metrics, and visualizes the results.

provides three views for exploring code clones: a *Statistics View*, a *Scatter Plot*, and a *Metric View*. To support these views, CC4M identifies microservice boundaries, detects Type-2 clone pairs, and enriches them with co-modification history and file-category information.

A. Microservice Identification

CC4M identifies microservices by applying CLAIM [11], a lightweight static analysis approach based on Docker Compose files and related Dockerfiles. CLAIM extracts service definitions from the Docker configuration files and estimates which directories correspond to application microservices.

CC4M then assigns each source file to a microservice based on the directory information obtained by CLAIM. This assignment is used to determine whether each clone pair is a within-service clone or a cross-service clone.

B. Code Clone Detection

CC4M identifies the programming languages used in the target repository using GitHub Linguist¹ and selects source files whose languages are supported by CCFinderSW.

Before running CCFinderSW, CC4M comments out import, include, and package declarations in a temporary analysis copy to exclude them from the token sequence used for clone detection and reduce trivial clones. The full list of filtered patterns is documented in our GitHub repository.²

CC4M then uses CCFinderSW [10] to obtain Type-2 clone pairs. Type-2 clones are clone pairs whose fragments are identical except for differences in whitespace, comments, and identifiers such as variable names and type names [5]. CCFinderSW supports multiple programming languages through ANTLR grammar definitions, which makes it suitable for heterogeneous microservice repositories.

C. Co-modification Analysis

CC4M identifies clone pairs that were modified together based on Git history. Users can select commits at a specified interval, merge commits, or commits associated with Git tags, depending on the desired analysis granularity.

For each selected target commit, CC4M performs clone detection and tracks code fragments across adjacent target commits using line correspondences obtained from `git diff`. For a tracked fragment, CC4M regards it as modified at commit c_i when lines in its corresponding range are added, deleted, or changed between c_{i-1} and c_i .

Let A and B be the two fragments forming a clone pair, and let C_A and C_B be the sets of target commits in which A and B were modified, respectively. CC4M regards the clone pair as co-modified when $C_A \cap C_B \neq \emptyset$, that is, when both fragments were modified in the same commit at least once.

D. Clone Pair Classification

CC4M annotates each clone pair with three kinds of information: service-boundary, co-modification, and file-category. Using the service mapping computed in Section II-A, it classifies each pair as either a within-service clone or a cross-service clone. Using the co-modification information computed in Section II-C, it records whether the pair has been co-modified in the Git history.

It also classifies each file into one of four categories: *Test*, *Config*, *Data*, or *Logic*. This classification is based on path and file-name patterns, and rules are applied in this priority order. It then assigns a category to each clone pair from the categories of the two files containing the cloned fragments. Clone pairs whose two files share the same category are categorized as *Test*, *Config*, or *Data*; pairs involving both test and non-test files are categorized as *Mixed*; and the remaining pairs are categorized as *Logic*. The full classification rules are documented in our GitHub repository³.

E. Clone Metrics Computation

The enriched clone pairs produced in Sections II-A to II-D carry rich attributes, making individual inspection impractical. To enable metric-based prioritization and filtering of the enriched clone pairs, CC4M characterizes them with clone metrics.

MS(s). This metric represents the set of microservices to which the code fragments in a given clone set s belong. Let $f \in s$ denote a code fragment in clone set s , and let

¹<https://github.com/github-linguist/linguist>

²https://github.com/gg117e/CC4M/blob/main/docs/declaration_filtering.md

³https://github.com/gg117e/CC4M/blob/main/docs/file_classification.md

$ms(f)$ denote the microservice to which fragment f belongs. Then, $MS(s) = \{ms(f) \mid f \in s\}$, and $|MS(s)|$ represents the number of microservices spanned by clone set s . A clone set s satisfying $|MS(s)| \geq 2$ is called a *cross-service clone set*.

$CM(s)$. This metric represents the set of co-modification commits for a given clone set s . A commit c is called a *co-modification commit* of s if at least two fragments in s are modified in c . Then, $CM(s)$ is defined as:

$$CM(s) = \{c \mid \text{at least two fragments in } s \text{ are modified in } c\}.$$

This extends the pair-level co-modification in Section II-C: $c \in CM(s)$ exactly when at least one clone pair within s is co-modified at c . $|CM(s)|$ represents the number of co-modification commits of s . The co-modification fragment ratio of s , denoted by $cmr(s)$, is defined as the fraction of fragments in s that are modified in at least one commit in $CM(s)$.

In the *Scatter Plot* and *Metric View* (see Section II-F), $|MS(s)|$, $|CM(s)|$, and $cmr(s)$ are shown as *service span*, *co-modification frequency*, and *co-modification fragment ratio*, respectively. In addition to these clone-set-level metrics, CC4M computes per-service and per-file metrics to support filtering and inspection, such as the number of cross-service clone sets, the ratio of cloned lines to service size, and the number of services shared through clone sets. The full definitions of these additional metrics are documented in our GitHub repository⁴.

F. Visualization

CC4M provides three views: (i) a *Statistics View* that reports clone statistics for the entire project, (ii) a *Scatter Plot* that visualizes the spatial distribution of clone pairs across service boundaries, and (iii) a *Metric View* that supports metric-driven filtering and drill-down across services, clone sets, and files.

Statistics View. It provides an overview of clones across the project. It summarizes the number of microservices, files, clone sets, and lines of code, breaks them down by language, and ranks the services that concentrate the most clones.

Scatter Plot. Fig. 2 shows a scatter plot in which the same set of source files is arranged on both axes in the same order and grouped by service. Gray dotted lines indicate service boundaries. A clone pair is plotted at the coordinates of the two files containing the cloned fragments; color intensity encodes overlapping clone pairs for the same file pair. Points inside diagonal service blocks represent within-service clones, whereas points outside represent cross-service clones. This layout helps users locate clone concentrations across services without inspecting individual file paths. A filter panel lets users restrict the displayed clone pairs by service scope, file category, and co-modification count. In Fig. 2, Service Scope is set to Cross, so the plot shows only cross-service clone pairs. Filters can also be combined; for example, restricting file-category to Logic and Co-modification Count to at least 1 surfaces cross-service Logic clones that have required coordinated changes in the past. Users can zoom or pan to inspect dense regions, and selecting a point opens a detail panel that

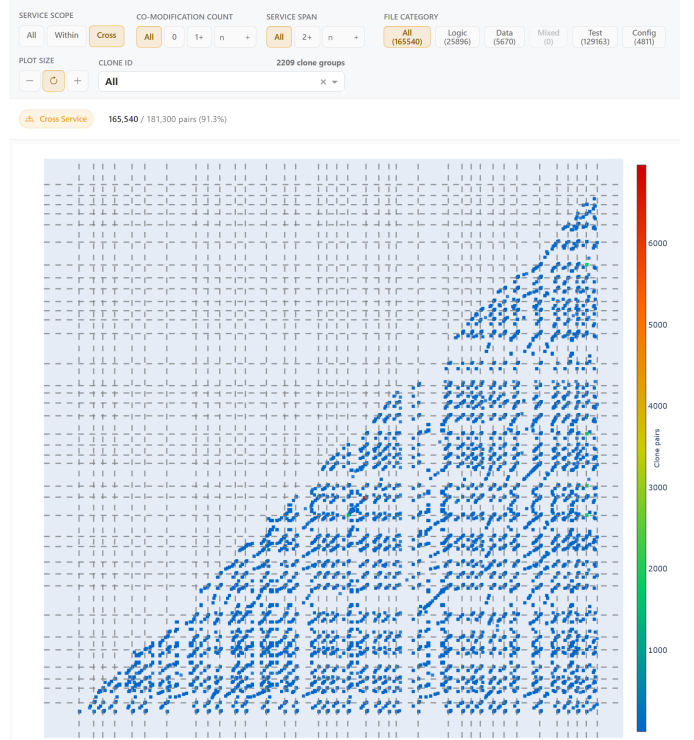


Fig. 2. A screenshot of the CC4M scatter plot generated for the TRAIN TICKET microservice project. The same source files are arranged on both axes and grouped by microservice, with gray dotted lines indicating service boundaries. The Service Scope filter is set to Cross, so the plot shows only cross-service clone pairs. Each point represents clones between a pair of files, and its color encodes the number of clone pairs.

shows the corresponding cloned fragments with surrounding source code.

Metric View. It supports metric-based prioritization and drill-down analysis of clone information. While the *Scatter Plot* helps users visually locate clone distributions across service boundaries, the *Metric View* helps users identify services, clone sets, and files that are likely to have larger maintenance impact according to the computed metrics. The *Metric View* provides three entry perspectives: Microservice-Based (MS-Based), Clone-Set-Based, and File-Based. In each perspective, users can narrow down the result list by combining multiple metric filters. In Clone-Set-Based, users can focus on clone sets that span multiple services or have past co-modifications by using metrics such as service span and co-modification frequency. In MS-Based, users can identify services that contain many cross-service clone sets or have a high cloned-line ratio. In File-Based, users can find files that share clones with many services or have cross-service co-modification histories. The filtered results are shown as ranked lists, allowing users to prioritize items before inspecting source code. When a clone set is selected, the *Metric View* displays the fragments contained in the clone set. Users can then select one or two fragments to view the corresponding source code. This staged workflow, from metric-based prioritization to code-level

⁴<https://github.com/gg117e/CC4M/blob/main/docs/metrics.md>

TABLE I
PER-LANGUAGE STATISTICS FOR TRAIN TICKET.

Language	Services	Files	LOC	Clone Sets	Clone Pairs
Go	1	2	57	0	0
Java	37	740	52,814	2,414	181,300
JavaScript	2	676	234,514	7,919	2,303,805
Python	1	5	360	0	0

inspection, helps users focus on clone relationships that are more likely to affect maintenance tasks.

III. USAGE SCENARIO

This section demonstrates how CC4M supports software evolution tasks using TRAIN TICKET [12], [13], an open-source microservice system selected from a curated benchmark dataset of Dockerized OSS microservice projects [14]. We selected TRAIN TICKET as the subject of the scenarios because it has been adopted as a benchmark in prior microservice research [4], [6], [15]. The system is built mainly in Java with a JavaScript frontend. We use the version of TRAIN TICKET at merge commit 813dd01⁵; all source-code references in this section refer to this version. For the scenarios in this section, we configure CC4M to detect co-modification at merge-commit granularity; that is, clone fragments are regarded as co-modified when they are modified in the same merge commit. Clone detection uses CCFinderSW with a minimum matching token length of 50 and import/package declaration filtering enabled. In this snapshot, CC4M identifies 41 microservices, 1,423 source files, 10,333 clone sets; Table I reports the per-language breakdown. The following three scenarios illustrate CC4M from different perspectives: Scenario 1 focuses on a duplicated booking workflow, Scenario 2 on broadly shared data clones with partial co-modification follow-up, and Scenario 3 on a repeatedly co-modified Account clone that may require further inspection. We also applied CC4M to six additional microservice systems selected from the microservice benchmark dataset [14]. Further details of these systems are available in our project documentation.⁶

Scenario 1. Consider a developer who reviews the system to understand where business-logic duplication concentrates across service boundaries. Without a specific target in mind, they start from the *Scatter Plot*, which gives a visual overview of which service pairs share heavy logic duplication. To narrow the view to such duplication, they apply the cross-service filter, the co-modification filter, and the Logic file-category filter. In the *Scatter Plot* view, a dark-red point indicates a file pair with many overlapping clone pairs; such points are useful starting points because they suggest concentrated logic duplication between two files. By clicking this point and selecting the clone set with the largest number of lines in the file pair, they identify the largest cross-service clone in the file pair. The two fragments are located in

PreserveServiceImpl.java⁷ of ts-preserve-service and PreserveOtherServiceImpl.java⁸ of ts-preserve-other-service. The clone set duplicates the tail of the booking workflow, including food order, consign, and notification processing. The *Metric View* shows service span $|MS(s)| = 2$, co-modification frequency $|CM(s)| = 1$, and co-modification fragment ratio $cmr(s) = 1.0$, meaning that both fragments were modified together in the same commit. This result suggests that future changes to the booking workflow may need to be applied in both services to avoid behavioral divergence.

Scenario 2. Consider a developer tasked with inspecting data definitions that may cause maintenance problems across service boundaries. In microservice systems, the same data entity can be duplicated in multiple services, and inconsistent updates to such copies may lead to subtle bugs. To find broadly shared data clones with incomplete follow-up, they filter clone sets by the Data file-category, apply the co-modification condition, and sort the results by service span in descending order and by co-modification fragment ratio in ascending order in the *Metric View*. This prioritizes data clone sets that are widely distributed but have incomplete co-modification follow-up. The top entry under these conditions is a clone set in which the same Order.java entity is duplicated across nine services; we cite the copies in ts-order-service⁹ and ts-preserve-service¹⁰ as representatives. The clone-set metrics show service span $|MS(s)| = 9$, co-modification frequency $|CM(s)| = 1$, and co-modification fragment ratio $cmr(s) = 0.67$. This indicates that only six of the nine fragments were modified in the past co-modification commit, leaving three services that did not follow the update. CC4M therefore makes partial follow-up explicit and helps them focus inspection on potentially inconsistent data definitions.

Scenario 3. Consider a developer who assesses the maintenance risk of the system. They look for cross-service clone sets that have repeatedly required coordinated changes, since these indicate a recurring maintenance burden. To surface them, they filter clone sets with a co-modification frequency of two or more in the *Metric View*. This filtering identifies a clone set in which the same Account.java entity is duplicated across three booking-related services: ts-preserve-service¹¹, ts-cancel-service¹², and ts-preserve-other-service¹³. The clone-set metrics show service span $|MS(s)| = 3$, co-modification frequency $|CM(s)| = 2$, and co-modification fragment ratio $cmr(s) = 1.0$. These metrics indicate that all three fragments were modified together twice. This repeated co-modification

⁷ts-preserve-service/src/main/java/preserve/service/PreserveServiceImpl.java:178-286

⁸ts-preserve-other-service/src/main/java/preserveOther/service/PreserveOtherServiceImpl.java:183-293

⁹ts-order-service/src/main/java/order/entity/Order.java:57-73

¹⁰ts-preserve-service/src/main/java/preserve/entity/Order.java:50-64

¹¹ts-preserve-service/src/main/java/preserve/entity/Account.java:1-38

¹²ts-cancel-service/src/main/java/cancel/entity/Account.java:1-38

¹³ts-preserve-other-service/src/main/java/preserveOther/entity/Account.java:1-38

⁵<https://github.com/FudanSELab/train-ticket/tree/813dd01>

⁶<https://github.com/gg117e/CC4M/blob/main/docs/dataset.md>

suggests that the Account entity may require further inspection, because future changes to the entity may again require coordinated updates across the three services. CC4M makes such clone sets discoverable by providing co-modification frequency as an explicit filter.

IV. RELATED WORK

Mo et al. [4] reported that code clones exist within and across microservices in open-source projects, and that some clones are co-modified in the same version. Zhao et al. [6] subsequently analyzed cross-service clones in 22 Java microservice projects and reported that 56.7% of cross-service clone pairs occur in data-related files. These findings indicate that co-modification and file category are useful perspectives for characterizing cross-service clones, and CC4M treats both as attributes of clone pairs.

For visualization, Baker introduced the dotplot to display duplicated code [16], and Gemini [7], the visualization front-end of CCFinder [17], places files on the two axes of a scatter plot to provide an overview of clone distributions. VisCad [8], SolidSDD [9], and the work of Choi et al. [18] support metric-based filtering and refactoring-candidate extraction from clone detection results. However, these existing approaches do not explicitly account for microservice boundaries or co-modification history across services.

Recent microservice visualization tools such as DENIM [19] and MicroKarta [20] visualize data access points or architectural structure to support software evolution, but do not target clone relationships. CC4M addresses this gap by overlaying explicit service boundaries on scatter-plot axes and by defining service-aware clone metrics such as service span and co-modification frequency.

V. LIMITATIONS

CC4M relies on Type-2 clone detection with CCFinderSW. Therefore, structurally similar but partially modified clones, known as Type-3 clones, fall outside its detection scope, and such cross-service clones may be missed. It also relies on CLAIM for service-boundary identification, so it applies only to microservice systems configured with Docker Compose and Dockerfiles. In addition, because CLAIM estimates service directories through static analysis, the estimated boundaries may contain errors. Prior work reports a microservice identification accuracy of 82.0% [11]. Such errors directly affect the classification of clone pairs as within-service or cross-service.

Finally, the clone metrics serve as prioritization aids rather than validated risk measures, because prior work reports that many clones are rarely changed and that inconsistent changes are infrequent [21]. CC4M therefore helps users focus inspection rather than automatically flagging clones as defects.

VI. CONCLUSION AND FUTURE WORK

This paper presented CC4M, a microservice-aware visualization tool for analyzing code clones. CC4M integrates microservice identification, Type-2 clone detection, co-modification analysis, file-category classification, and clone

metrics, and visualizes the enriched clone data with explicit service boundaries. Future work will extend CC4M to Type-3 clones and evaluate CC4M's usefulness with developers performing microservice maintenance tasks.

VII. ACKNOWLEDGMENT

Portions of the implementation were made with the assistance of Claude Code. Claude and ChatGPT were also used for English translation and proofreading. All content was reviewed and validated by the authors.

REFERENCES

- [1] C. Richardson, *Microservices Patterns: with Examples in Java*. Simon and Schuster, 2018.
- [2] S. Newman, *Building Microservices*. O'Reilly Media, Inc., 2021.
- [3] J. Lewis and M. Fowler, "Microservices," <https://martinfowler.com/articles/microservices.html>, Mar. 2014, accessed: 8 December 2025.
- [4] R. Mo, Y. Zhao, Q. Feng, and Z. Li, "The existence and co-modifications of code clones within or across microservices," in *Proc. of ESEM*, 2021, pp. 22:1–22:11.
- [5] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Sci. Comput. Program.*, vol. 74, no. 7, pp. 470–495, 2009.
- [6] Y. Zhao, R. Mo, Y. Zhang, S. Zhang, and P. Xiong, "Exploring and understanding cross-service code clones in microservice projects," in *Proc. of ICPC*, 2022, pp. 449–459.
- [7] Y. Ueda, T. Kamiya, S. Kusumoto, and K. Inoue, "Gemini: Maintenance support environment based on code clone analysis," in *Proc. of METRICS*, 2002, pp. 67–76.
- [8] M. Asaduzzaman, C. K. Roy, and K. A. Schneider, "Viscad: Flexible code clone analysis support for nicad," in *Proc. of IWSC*, 2011, pp. 77–78.
- [9] L. Voinea and A. C. Telea, "Visual clone analysis with solidsdd," in *Proc. of VISSOFT*, 2014, pp. 79–82.
- [10] Y. Semura, N. Yoshida, E. Choi, and K. Inoue, "CCFinderSW: Clone detection tool with flexible multilingual tokenization," in *Proc. of APSEC*, 2017, pp. 654–659.
- [11] K. Maggi, R. Verdecchia, L. Scommegna, and E. Vicario, "Claim: a lightweight approach to identify microservices in dockerized environments," in *Proc. of EASE*, 2024, pp. 357–362.
- [12] X. Zhou, X. Peng, T. Xie, J. Sun, C. Xu, C. Ji, and W. Zhao, "Benchmarking microservice systems for software engineering research," in *Proc. of ICSE Companion*, 2018, pp. 323–324.
- [13] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, 2021.
- [14] D. A. d'Aragnona, A. Bakhtin, X. Li, R. Su, L. Adams, E. Aponte, F. Boyle, P. Boyle, R. Koerner, J. Lee, F. Tian, Y. Wang, J. Nyyssölä, E. Quevedo, S. M. Rahaman, A. S. Abdelfattah, M. Mäntylä, T. Cerny, and D. Taibi, "A dataset of microservices-based open-source projects," in *Proc. of MSR*, 2024, pp. 504–509.
- [15] T. Cerny, G. Goulis, and A. S. Abdelfattah, "Towards change impact analysis in microservices-based system evolution," in *Proc. of SANER*, 2025.
- [16] B. S. Baker, "A program for identifying duplicated code," in *Computer Science and Statistics*, 1992, pp. 49–57.
- [17] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A multilingual token-based code clone detection system for large scale source code," *IEEE Trans. Softw. Eng.*, vol. 28, no. 07, pp. 654–670, jul 2002.
- [18] E. Choi, N. Yoshida, T. Ishio, K. Inoue, and T. Sano, "Extracting code clones for refactoring using combinations of clone metrics," in *Proc. of IWSC*, 2011, pp. 7–13.
- [19] M. André, M. Raglianti, A. Cleve, and M. Lanza, "DENIM: Exploring data access in microservices," in *Proc. of ICSME*, 2025, pp. 900–904.
- [20] O. Manglaras, A. Farkas, P. Fule, C. Treude, and M. Wagner, "MicroKarta: Visualising microservice architectures," in *Proc. of FSE Companion*, 2024.
- [21] N. Göde and R. Koschke, "Frequency and risks of changes to clones," in *Proc. of ICSE*, 2011, pp. 311–320. [Online]. Available: <https://doi.org/10.1145/1985793.1985836>